

REPORT:

FOOD ORDER MANAGEMENT SYSTEM

USING SINGLY LINKED LIST

Submitted by:

Jyoti Gavhane

Department of Electronics and Telecommunication Engineering

Academic Year: 2026–27

1. INTRODUCTION

The Food Order Management System is a simple console-based application developed in C++ using a Singly Linked List.

The system is designed to manage food orders dynamically.

Each food order is stored as a node in the linked list.

Each node contains the Order ID, Customer Name, Food Name, and a pointer to the next node.

New orders are added at the end of the linked list. When an order is completed, the first order is removed from the list.

This project demonstrates the practical application of the Singly Linked List data structure in a real-world food order management system.

2. OBJECTIVES

The main objectives of this project are:

1. To understand the concept of Singly Linked List.
2. To implement insertion, traversal and deletion operations.
3. To understand pointers and dynamic memory allocation.
4. To develop a simple real-world application using Linked List.
5. To manage food orders dynamically.
6. To understand how data structures can be used to solve practical problems.

3. PROBLEM STATEMENT

Design and implement a console-based Food Order Management System in C++ using a Singly Linked List.

The system should allow the user to:

1. Add a new food order.
2. Display all pending food orders.
3. Complete the first pending order.
4. Remove the completed order from the linked list.
5. Exit the system.

Each food order is stored as a node containing Order ID, Customer Name, Food Name and a pointer to the next node.

New orders are inserted at the end of the linked list and completed orders are deleted from the beginning.

4. DATA STRUCTURE USED

Data Structure Used: Singly Linked List

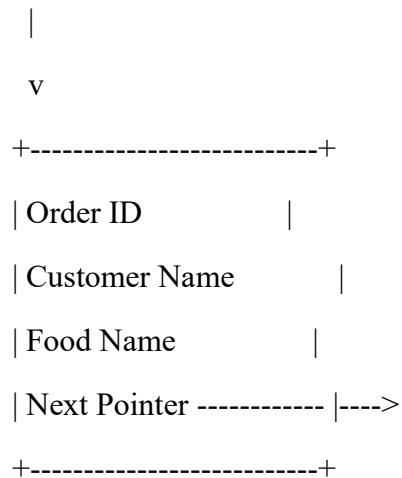
A Singly Linked List consists of nodes connected using pointers.

Each node in this project contains:

- Order ID
- Customer Name
- Food Name
- Next Pointer

Node representation:

HEAD



Example:

HEAD → [101 | Jyoti | Pizza] → [102 | Rahul | Burger]
→ [103 | Sneha | Biryani] → NULL

5. NODE STRUCTURE

The node is represented using a structure in C++.

Each node contains three data fields and one pointer:

1. Order ID – stores the unique identification number.
2. Customer Name – stores the name of the customer.
3. Food Name – stores the ordered food item.
4. Next – stores the address of the next node.

The structure is:

```
struct Node
{
    int orderID;
    string customerName;
    string foodName;
    Node* next;
};
```

6. OPERATIONS PERFORMED

The system performs the following operations:

1. INSERTION

A new food order is inserted at the end of the linked list.

Example:

Before:

HEAD → Order 101 → Order 102 → NULL

After adding Order 103:

HEAD → Order 101 → Order 102 → Order 103 → NULL

2. TRAVERSAL / DISPLAY

The linked list is traversed from HEAD to NULL.

Each node is visited and its order details are displayed.

Example:

HEAD → Order 101 → Order 102 → Order 103 → NULL

3. DELETION

When the first order is completed, the first node is deleted.

Before deletion:

HEAD → Order 101 → Order 102 → Order 103 → NULL

After completing Order 101:

HEAD → Order 102 → Order 103 → NULL

4. EXIT

The program terminates when the user selects the Exit option.

7. WORKING OF THE SYSTEM

Initially, the linked list is empty.

HEAD → NULL

When the first order is added:

HEAD → [101 | Pizza] → NULL

When the second order is added:

HEAD → [101 | Pizza] → [102 | Burger] → NULL

When the third order is added:

HEAD → [101 | Pizza] → [102 | Burger] → [103 | Biryani]
→ NULL

When the first order is completed, the HEAD pointer moves to the second node and the first node is deleted.

HEAD → [102 | Burger] → [103 | Biryani] → NULL

Thus, the system maintains the pending food orders dynamically.

8. ALGORITHM

Step 1: Start the program.

Step 2: Initialize HEAD = NULL.

Step 3: Display the main menu.

Step 4: Accept the user's choice.

Step 5: If the user selects Add New Order, create a new node.

Step 6: Enter Order ID, Customer Name and Food Name.

Step 7: If the list is empty, make the new node the HEAD.

Step 8: Otherwise, traverse to the last node and connect the
new node to it.

Step 9: If the user selects Display Orders, traverse the list
from HEAD to NULL and display all orders.

Step 10: If the user selects Place / Complete Order, check
whether the list is empty.

Step 11: If the list is not empty, store the first node in a
temporary pointer.

Step 12: Move HEAD to the next node.

Step 13: Delete the previous first node.

Step 14: Repeat the menu until the user selects Exit.

Step 15: Stop the program.

9. FLOW OF THE SYSTEM

START

|

v

Initialize HEAD = NULL

|

v

Display Menu

|

v

Enter Choice

|

+-----+

|

v

|

v

Add Order Display Orders

| |

v v

Insert at End Traverse List

| |

+-----+-----+

|

v

Complete Order

|

v

Delete First Node

|

v

Check Exit

/ \

No Yes

| |

v v

Menu STOP

10. ADVANTAGES

1. Simple and easy to understand.

2. Uses dynamic memory allocation.
3. No fixed number of orders is required.
4. New orders can be added dynamically.
5. Completed orders can be easily removed.
6. Demonstrates practical use of pointers.
7. Helps in understanding Singly Linked List operations.
8. The system can be extended with additional features in the future.

11. APPLICATIONS

The concept used in this project can be applied to:

1. Food ordering systems.
2. Restaurant order management.
3. Queue management systems.
4. Customer service systems.

5. Ticket booking systems.
6. Task management systems.
7. Print job management.
8. Service request management.

12. PROGRAMMING LANGUAGE AND TOOLS

Programming Language: C++

Data Structure: Singly Linked List

Concepts Used:

- Structures
- Pointers
- Dynamic Memory Allocation
- Linked List
- Insertion
- Traversal
- Deletion
- Functions
- Conditional Statements
- Loops
- Switch Case

13. RESULT

The Food Order Management System was successfully implemented in C++ using a Singly Linked List.

The system successfully performs the following operations:

- Adding new food orders
- Displaying pending orders
- Completing the first order
- Deleting the completed order
- Maintaining the linked list dynamically

The project successfully demonstrates the practical implementation of insertion, traversal and deletion operations using a Singly Linked List.

14. CONCLUSION

The Food Order Management System provides a simple

real-world application of the Singly Linked List data structure.

Through this project, the concepts of nodes, pointers, dynamic memory allocation, insertion, traversal and deletion were understood and implemented.

New food orders are inserted at the end of the linked list, while completed orders are removed from the beginning.

The project shows how data structures can be used to efficiently manage dynamically changing information.

Thus, the Food Order Management System successfully demonstrates the practical use of a Singly Linked List in C++.

15. FUTURE SCOPE

The current system can be further improved by adding:

1. Food price and total bill calculation.
2. Customer contact details.

3. Different food categories.
4. Order cancellation.
5. Search order by Order ID.
6. Update order details.
7. Saving orders into a file or database.
8. Graphical User Interface.
9. Online food ordering functionality.
10. Payment management.

16. REFERENCES

1. Data Structures and Algorithms using C++ concepts.
2. C++ programming concepts including structures, pointers and dynamic memory allocation.

3. Singly Linked List concepts and operations studied in
Data Structures and Algorithms.

17. PROJECT OUTCOME

The project helped in understanding how a Singly Linked List can be used to solve a practical problem.

It also improved understanding of dynamic memory allocation, pointers, node creation, insertion, traversal and deletion.

The project provides a foundation for developing more advanced food ordering and management applications in the future.

--- END ---